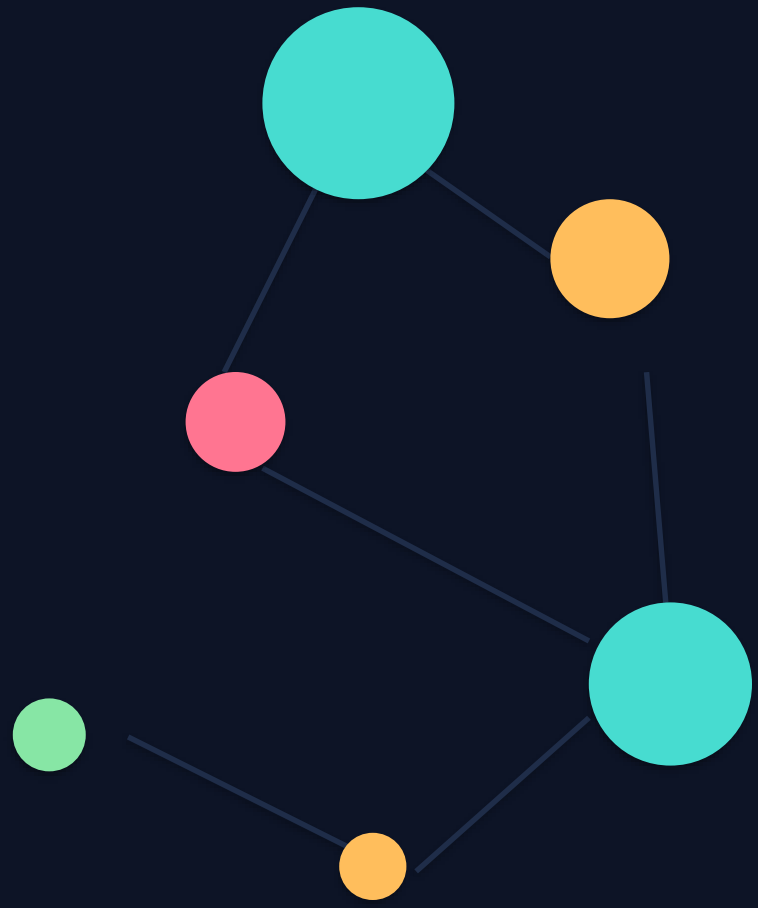
Глубокое обучение с подкреплением

Павлов Михаил • Виднов Максим



Когда нужно учиться действовать, а не просто предсказывать?

Ключевой вопрос: какое действие приведёт к лучшему результату в долгосрочной перспективе?

Примеры задач

- Робот: двигаться и не сталкиваться
- Игра: максимизировать итоговый счёт
- Энергосистема: распределять нагрузку
- Рекомендации: учитывать отложенную реакцию

Почему сложно задать правила?

- Слишком много возможных состояний
- Действия меняют будущие возможности
- Награда может прийти не сразу
- Среда бывает неизвестной или меняющейся

Что такое глубокое обучение с подкреплением?

Deep Reinforcement Learning (Deep RL)

Обучение с подкреплением

Учимся выбирать действия по награде и опыту

Глубокая нейросеть

Приближает политику или ценность состояний

Вместе

Учимся в сложных пространствах состояний

Не магия: алгоритм улучшает стратегию, пробуя действия и оценивая последствия.

Цикл взаимодействия агента со средой

На каждом шаге агент наблюдает, действует и получает обратную связь

1. Состояние s_t

Что сейчас видит агент

→

2. Агент выбирает a_t

Какое действие выполнить

→

3. Среда отвечает

Новое состояние s_(t+1) и награда r_(t+1)

Опыт: (состояние, действие, награда, новое состояние)

Агент повторяет цикл много раз и постепенно улучшает стратегию.

Пять элементов задачи

Обычно задачу описывают как марковский процесс принятия решений (MDP)

S · Состояния	Описание ситуации
A · Действия	Что агент может сделать
P · Переходы	Как меняется среда
R · Награда	Численная оценка результата
γ · Дисконтирование	Насколько важны будущие награды

Цель: максимизировать ожидаемую сумму наград, а не только мгновенный выигрыш.

Почему важны отложенные награды?

Вознаграждение за шаг и итоговая цель — не всегда одно и то же

Жадное решение

Выбрать действие с максимальной наградой прямо сейчас

Стратегическое решение

Выбрать действие, которое улучшает суммарный результат

$$G_t = r_{t+1} + \gamma * r_{t+2} + \gamma^2 * r_{t+3} + \dots$$

Что именно предсказывает нейросеть?

Есть несколько распространённых подходов

Q-сеть

Q(s, a): насколько выгодно действие a в состоянии s

Например: DQN

Политика

π(a | s): вероятность выбрать действие

Например: PPO, A2C

Модель среды

Предсказывает будущие состояния и награды

Model-based RL

Нейросеть помогает обобщать опыт на похожие состояния, которых агент ещё не встречал.

DQN: Q-learning с нейросетью

Deep Q-Network — классический пример Deep RL

- Сеть получает состояние s и выдаёт Q(s, a) для действий
- Выбираем действие: иногда лучшее, иногда случайное (ε-greedy)
- Сохраняем переходы в буфер воспроизведения
- Обучаем сеть приближать целевое Q-значение
- Целевая сеть стабилизирует обучение

Целевое значение

$$y = r + \gamma \cdot \max_{a'} Q_target(s', a')$$

Ошибка: $(y - Q(s,a))^2$

Мини-лабиринт: идти к цели

Состояния: A (старт), B (развилка), G (цель). Два действия: влево или вправо.



Правило: шаг к цели даёт +10, обычный шаг −1, γ = 0,9.

Шаг 1. Обновляем оценку действия

Формула: Qновое = Qстарое + α × (цель − Qстарое)

Что известно

Q(B, вправо) = 2
Награда r = +10
max Q(G, ·) = 0

Подставляем значения

α = 0,5; γ = 0,9
Цель = 10 + 0,9 × 0 = 10
Ошибка = 10 − 2 = 8

Q(новое) = 2 + 0,5 × 8 = 6

Шаг 2. Награда распространяется назад

Обновим оценку перехода A → B, который ведёт к перспективному состоянию B.

До обновления

Q(A, вправо) = 0

r = −1

max Q(B, ·) = 6

Расчёт по формуле

Цель = −1 + 0,9 × 6 = 4,4

Qновое = 0 + 0,5 × 4,4 = 2,2

Шаг с мгновенным штрафом −1 полезен, потому что ведёт к цели.

Q-learning: инициализация и среда

Сначала задаём награды и правила переходов между состояниями.

```
import random

# 0 = A, 1 = B, 2 = цель
# действия: 0 = влево, 1 = вправо
Q = [[0.0, 0.0] for _ in range(3)]
alpha, gamma, epsilon = 0.5, 0.9, 0.15

def step(state, action):
    if state == 0:  # A
        return (1, -1) if action == 1 else (0, -2)
    if state == 1:  # B
        return (2, 10) if action == 1 else (0, -1)
    return (2, 0)
```

Q-learning: выбор действия и переход

```
for episode in range(500):
    s = 0
    for _ in range(10):
        if random.random() < epsilon:
            a = random.randrange(2)  # исследуем
        else:
            a = max(range(2), key=lambda x: Q[s][x])
        ns, r = step(s, a)
        # Цель достигнута — будущей награды больше нет
        target = r if ns == 2 else r + gamma * max(Q[ns])
```

Q-learning: обновление и вывод результата

```
        # Сдвигаем оценку к целевому значению
        Q[s][a] += alpha * (target - Q[s][a])
        s = ns
        if s == 2:
            break

print("Q(A):", Q[0], "Q(B):", Q[1])
print("Лучшее действие из A:",
      "вправо" if Q[0][1] > Q[0][0] else "влево")
```

Как запустить и интерпретировать результат

Пример использует только стандартную библиотеку Python 3.

Запуск

1. Сохранить три фрагмента подряд в файл q_learning.py
2. Запустить: python q_learning.py
3. Числа немного различаются из-за случайных действий.

Что проверить

Q(B, вправо) растёт, потому что ведёт к +10.

Q(A, вправо) растёт, потому что ведёт к B и далее к цели.

Важно: это табличный Q-learning; DQN использует нейросеть вместо таблицы.

Как превратить пример в DQN?

В большой задаче таблица Q слишком велика — её заменяет нейросеть

Табличный Q-learning

- Хранит отдельную оценку для каждой пары (s, a)
- Подходит для малых дискретных задач
- Не обобщает на похожие состояния

DQN

- Вход: признаки или изображение состояния
- Выход: оценки Q для возможных действий
- Обучение весов сети по TD-ошибке

Идея обновления та же; меняется способ представления функции Q.

Какие есть другие подходы?

Выбор зависит от размера пространства, доступности модели среды и типа действий

Подход	Когда использовать	Плюс	Минус
Табличный Q-learning	Малые дискретные задачи	Прост, интерпретируем	Плохо масштабируется
DQN	Дискретные действия, сложные наблюдения	Хорош для игр/состояний с большим числом признаков	Чувствителен к настройкам
Policy Gradient / PPO	Дискретные и непрерывные действия	Оптимизирует политику напрямую	Может требовать много опыта
Actor–Critic	Политика + оценка ценности	Часто эффективен и гибок	Сложнее в реализации
Model-based RL	Можно моделировать динамику	Планирование экономит реальные пробы	Ошибки модели вредят

Краткая сравнительная таблица

Нет универсально лучшего алгоритма

Критерий	Табличный RL	DQN	PPO / Actor–Critic	Model-based
Непрерывные действия	Обычно нет	Обычно нет	Да	Да, зависит от метода
Большие состояния	Нет	Да	Да	Да, зависит от модели
Простота	Высокая	Средняя	Средняя	Низкая/средняя
Потребность в данных	Низкая в игрушечных задачах	Часто высокая	Часто высокая	Может быть ниже
Типичный риск	Нет обобщения	Нестабильность	Высокая дисперсия	Смещение модели

Почему Deep RL трудно применять?

Высокая выразительность не гарантирует надёжного решения

- Данные дороги: агенту часто нужны тысячи или миллионы шагов
- Награду сложно спроектировать — возможна оптимизация не той цели
- Обучение нестабильно и чувствительно к гиперпараметрам
- Исследование опасно в реальном мире: нужны симуляторы и ограничения
- Результат может плохо переноситься в новые условия

Практический вывод

Сначала — симуляция, метрики и безопасные ограничения.

Где это используют — и где осторожничают

Лучше всего подходит, когда решения влияют на будущие состояния

Игры и симуляции

Безопасно генерировать большое число эпизодов

Робототехника

Манипуляции, движение, управление

Ресурсы и энергия

Распределение нагрузки и управление системами

Рекомендации / операции

Последовательные решения с отложенным эффектом

Для критичных систем обычно нужны дополнительные гарантии безопасности и проверки.

ИТОГИ

Три мысли, которые стоит запомнить

01	**Цель — долгосрочная награда**	Агент учится на последствиях действий, а не на готовых ответах.
02	**Нейросеть масштабирует представление**	Она приближает Q-функцию или политику для сложных состояний.
03	**Метод выбирают под задачу**	Сравнивают стоимость данных, стабильность, тип действий и безопасность.

Вопросы?